

Name: _____

In-class Quiz #07: Special Purpose Registers

Give all answers in **hexadecimal**. Refer to the code provided for this ICQ and the image on the following to answer the question.

Note. Because we have declared 3 variables before the Stack, the initial (empty) position of the Stack Pointer may be different from the usual one of 0x20000400. You can refer to the code or the image for the initial position.

1. What is the hexadecimal address of the next instruction to be executed as shown in the image?

Address: 0x_____

2. Is the stack current empty? (Circle the correct answer.)

Yes No

3. What is the machine code (hexadecimal) for the instruction Line 49 `PUSH {R0, R1}`?

ARM machine code: 0x_____

4. At what memory address can we find the machine code of the instruction Line 51 `POP {R0, R1}`?

Address: 0x_____

5. What will be the value stored (hexadecimal) in the PC immediately after the instruction on line 49 is executed? (`PUSH {R0, R1}`)

Address: 0x_____

6. What will be the value stored (hexadecimal) in the Stack immediately after the instruction on line 49 is executed? (`PUSH {R0, R1}`). If the contents at an address can NOT be determined, leave it blank.

Address	Contents (a cell means a byte)			
0x20000404				
0x20000408				
0x2000040C				
0x20000410				
0x20000414				

7. What will be the value stored (hexadecimal) in the SP immediately after the instruction on line 49 is executed? (PUSH {R0, R1})

Address: 0x_____

8. What will be the value stored (hexadecimal) in the LR immediately after the instruction on line 50 is executed? (BL addemup)

Address: 0x_____

9. What will be the value stored in the SP immediately after the instruction on Line 51 is executed? (POP {R0, R1})

Address: 0x_____

10. If the Line 61 and Line 62 are uncommented (will run these two lines), what will be the value stored in register R0 and R1 immediately after run Line 51 POP {R0, R1} .

R0: 0x_____

R1: 0x_____

The screenshot displays a microprocessor development tool interface. On the left, the 'Registers' panel shows the state of various registers. The 'Core' section lists registers R0 through R15, with R15 (PC) highlighted at address 0x000029C. Below this, the 'xPSR' register is shown at 0x01000000. The 'System' and 'Internal' sections are also visible, showing fields like Mode, Privilege, Stack, States, and Sec. The 'FPU' section is at the bottom.

The main 'Disassembly' window shows the assembly code for the current address range. The code includes instructions like LDR, PUSH, BL, and POP, with comments and addresses. The current instruction being executed is at address 0x000029C: `PUSH {R0,R1}`.

At the bottom, the 'main.s' file is open, showing the source code for the program. The code includes comments and instructions like `LDR R1, =VarY`, `MOV R0, #0x2222`, `STR R0, [R1]`, `LDR R2, =VarX`, `LDR R0, [R2]`, `LDR R3, =VarY`, `LDR R1, [R3]`, `PUSH {R0,R1}`, `BL addemup`, `POP {R0,R1}`, `B endf`, `;;; Subroutines`, `addemup`, `ADD R0, R0, R1`, `LDR R1, =VarZ`, `STR R0, [R1]`, `; LDR R1, =0x2000040C`, `; STR R0, [R1]`, `BX LR`, `endf`, `NOP`, and `NOP`.